Il Processore o CPU

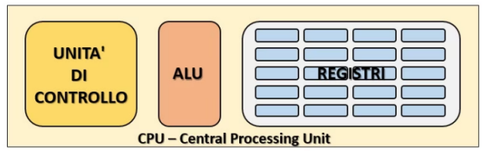
Architettura degli elaboratori AA 2023/24Corso di Laurea Triennale in Informatica

Massimo Orazio Spata
massimo.spata@unict.it
Dipartimento di Matematica e Informatica



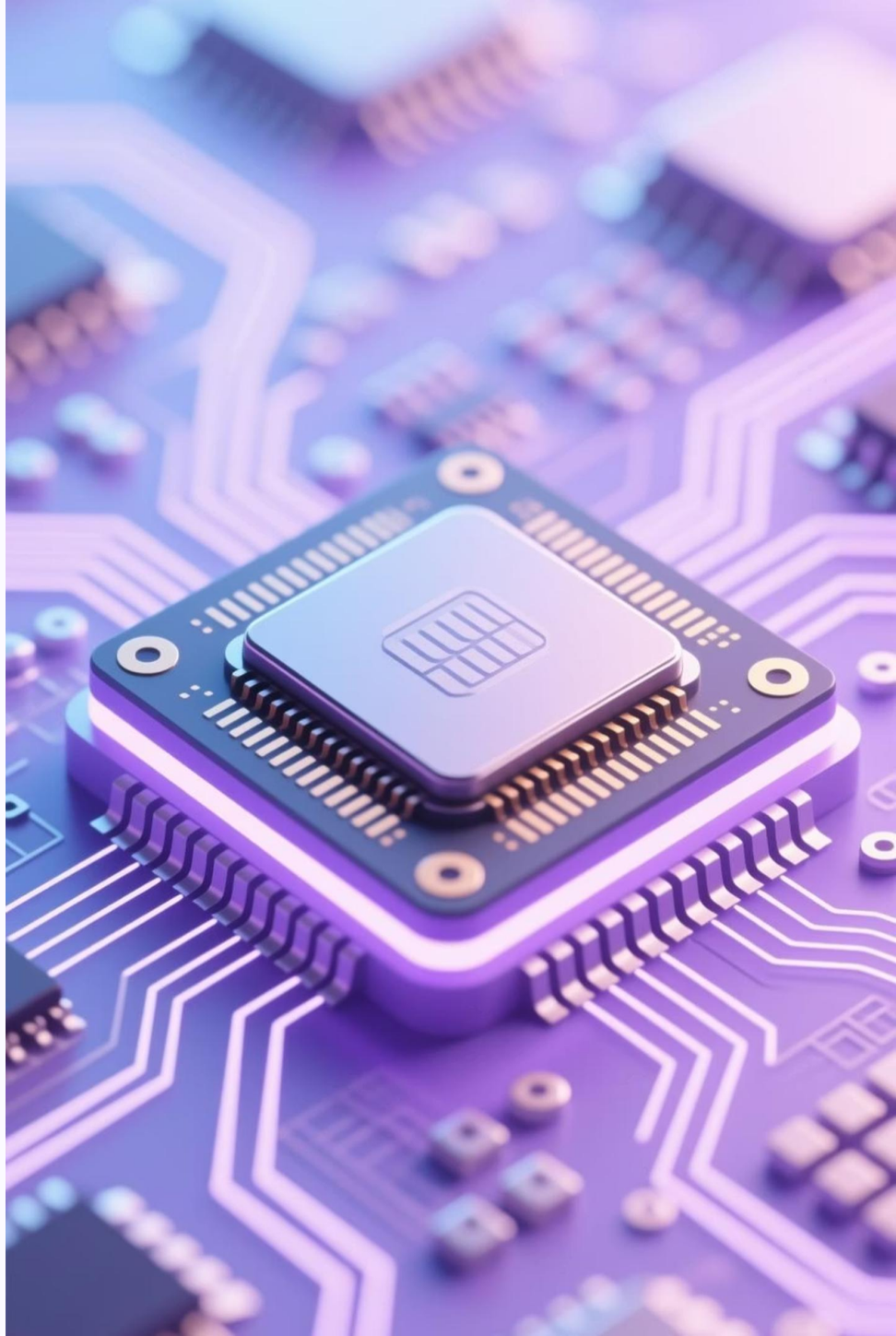
Componenti di un Processore

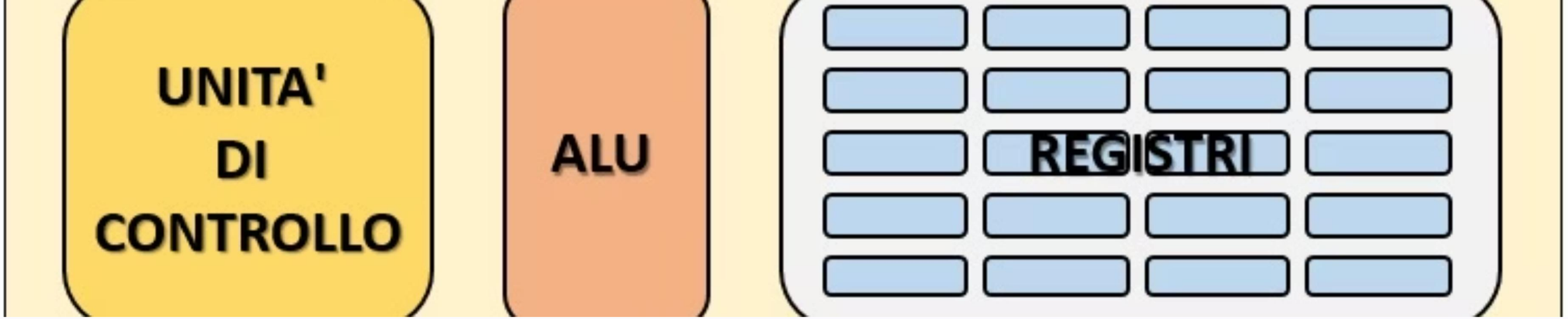
Un processore è un singolo circuito integrato in grado di effettuare **operazioni decisionali, di calcolo o di elaborazione dell'informazione**. Spesso il processore è indicato con la sigla **CPU** (Central Processor Unit).



Il processore può essere visto come suddiviso in **tre unità funzionali**: la **CU** (unità di controllo), l'**area dei registri** e l'**ALU** (unità aritmetico-logica).

La CU si affaccia sul bus, lo arbitra impostando i valori sulle linee ABus, DBus e CBus. Legge il DBus e il CBus, legge dalla memoria (e dall'I/O) i dati o li aggiorna dopo aver compiuto operazioni.





Registri e ALU

I Registri

Contengono i dati letti dall'UC sul bus per predisporli all'esecuzione delle istruzioni che avverranno nell'ALU. Oppure, contengono i risultati delle operazioni compiute dall'ALU in attesa di essere passati all'UC e quindi sul bus.

L'ALU

È l'unità di esecuzione effettiva del processore. Al suo interno si trovano **microprogrammi cablati** direttamente in hardware, scritti nel cosiddetto **microcodice** con relative microistruzioni.



Instruction Set Architecture

Ogni processore viene progettato con un **set di istruzioni specifico** denominato **ISA** (Instruction Set Architecture o Instruction Set). In corrispondenza di ognuna delle istruzioni è implementato un preciso microprogramma in ALU.

Op. Code

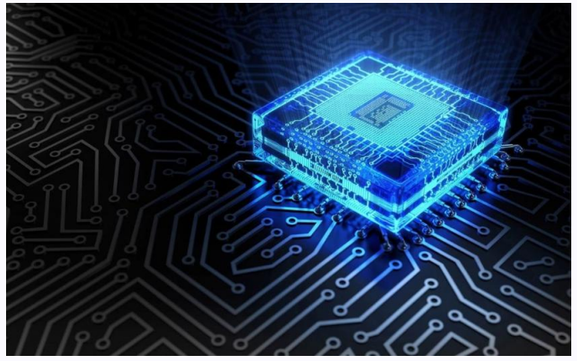
Ogni istruzione dell'ISA è contraddistinta da un numero specifico, denominato **Op. Code** (Operation Code). Ogni istruzione necessita di un numero preciso di parametri (operandi).

Codice Mnemonico

Ad ogni Op. Code si associa una breve descrizione in lingua naturale che ne ricorda la funzione, detta **codice mnemonico**.

Program Counter

Un registro speciale del processore, detto **PC** (Program Counter), si incrementa automaticamente della lunghezza dell'istruzione appena eseguita.



Il Data Path

Struttura del Data Path

L'**ALU**, i **registri** e molti **bus** costituiscono il data path.

- L'ALU può avere **2 registri di input** e **1 di output**
- I registri sono collegati ai registri di input
- Il registro di output è collegato ai registri



Ciclo del data path: passaggio di due operandi (A e B) attraverso la ALU e memorizzazione del risultato (A + B) nei registri.



Tipologie di Istruzioni

Registro-Memoria

Trasferisce un dato dalla memoria esterna ai registri, o viceversa. Permette lo scambio di informazioni tra il processore e la memoria principale.

Registro-Registro

Trasferisce un dato da un registro a un altro all'interno del processore. Operazioni molto veloci che non richiedono accesso alla memoria.

Ciclo del data path: passaggio di due operandi (*A* e *B*) attraverso la ALU e memorizzazione del risultato (*A* + *B*) nei registri.



Architetture CISC vs RISC

Le istruzioni possono essere implementate in due modi fondamentali:

Via Hardware

Implementazione diretta nella CPU per massime prestazioni

Via Software

Tramite un interprete implementato secondo il metodo hardware

CISC

Complex Instruction Set Computer

- Sostenute da IBM, VAX, Intel
- **Molte** e **complicate** istruzioni
- Istruzioni **interpretate**
- Maggiore complessità hardware

RISC

Reduced Instruction Set Computer

- Nate negli anni '80 (DEC Alpha)
- **Poche** e **semplici** istruzioni
- Istruzioni **non interpretate**
- Maggiore efficienza

Oggi, anche il **Pentium** è ibridato con tecnologia *RISC*, combinando i vantaggi di entrambe le architetture.

Il Ciclo Fetch-Decode-Execute-Store

01

Fetch

L'UC pone sull'ABus il valore del registro **Program Counter**, imposta lettura da memoria e carica l'**Op. Code** ivi memorizzato.

02

Decode

L'UC, a partire dall'Op.Code, determina la **lunghezza dell'istruzione** e la quantità di parametri necessari. Segue una fase di *Operand Fetch* per caricare gli operandi negli indirizzi adiacenti.

03

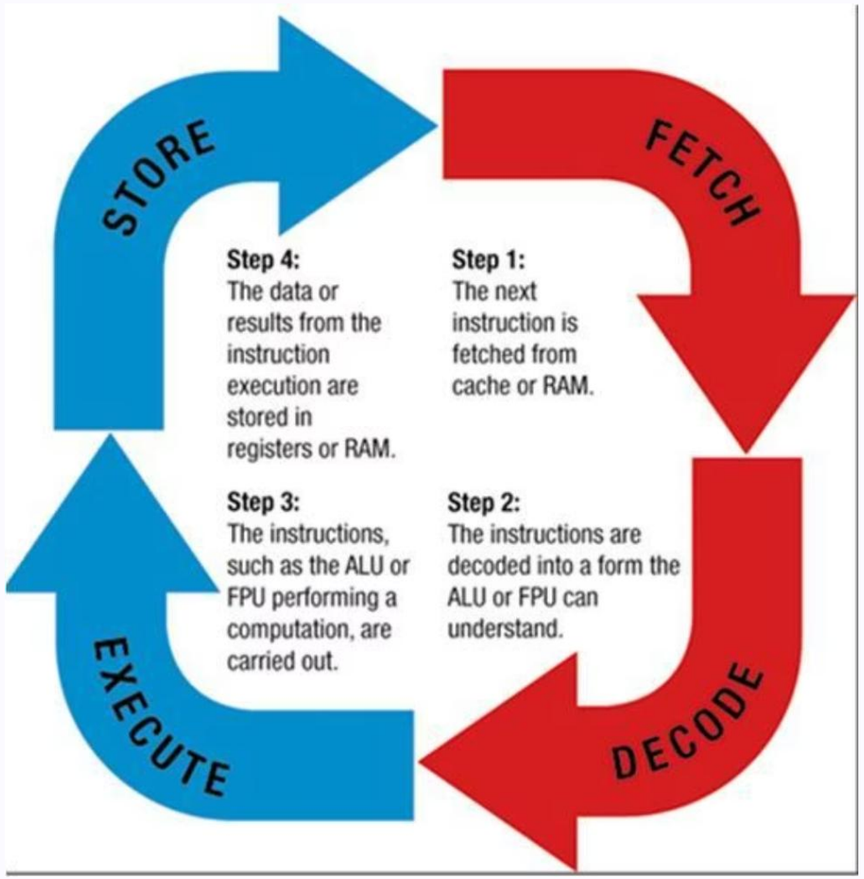
Execute

Viene avviato il **microprogramma** relativo all'Op. Code attuale, che usa i parametri memorizzati nei registri. La frequenza è regolata dal **clock di CPU**.

04

Store

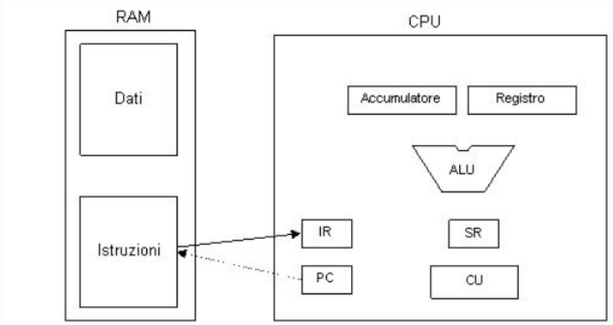
Al termine dell'Execute, i risultati nei registri vengono scritti dall'UC attraverso il bus verso la memoria o verso l'I/O.



Il processore ripete questa rigida sequenza di passi **fino all'arresto della macchina**, garantendo l'esecuzione ordinata delle istruzioni del programma.

Gestione delle Interruzioni

Il ciclo del processore termina consultando il **segnale INTR** per verificare se il controller di una periferica ha richiesto un'interruzione.



Un'**interruzione** rappresenta la sospensione temporanea dell'esecuzione del programma corrente per servire il codice associato alla interruzione pendente.

Questo meccanismo permette al sistema di gestire eventi esterni in modo efficiente, come l'input da tastiera, i dati dalla rete o il completamento di operazioni su disco.



Tempo di Esecuzione delle Istruzioni

Caratteristiche di un'Istruzione ISA

Ogni singola istruzione dell'ISA di un processore è contraddistinta da:

- Un proprio **Op. Code** univoco
- Una determinata **lunghezza** in byte (basata sul numero di operandi)
- Un preciso **numero di cicli di bus** per il completamento (compresi fetch, decode, execute e store)

Fattori di Performance

Il tempo di effettiva esecuzione del microprogramma influisce **relativamente** sulla durata dell'istruzione.

Questo perché il **clock di CPU** è di almeno un **ordine di grandezza superiore** al clock di bus, rendendo le operazioni interne molto più veloci degli accessi alla memoria.

La velocità del processore dipende quindi non solo dalla frequenza del clock, ma anche dall'efficienza dell'architettura e dal numero di cicli richiesti per ogni istruzione.

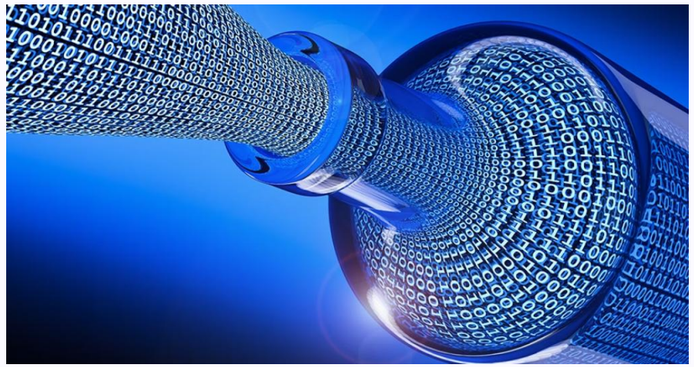
Il Bottleneck dell'Architettura CISC

Il concetto di collo di bottiglia nell'architettura dei microprocessori rappresenta una delle sfide fondamentali nella progettazione di sistemi di calcolo efficienti. Questa problematica è particolarmente evidente nelle architetture CISC (Complex Instruction Set Computer), dove la fase di decodifica delle istruzioni costituisce un punto critico che limita significativamente le prestazioni complessive del sistema.

Cause del Bottleneck CISC

Il bottleneck nelle architetture CISC è causato principalmente da una **fase di decode estremamente lunga e onerosa**. Questo rallentamento deriva dalla natura stessa dell'ISA (Instruction Set Architecture) CISC, caratterizzato da:

- Un'elevata quantità di istruzioni disponibili nel set
- Complessità variabile delle singole istruzioni
- Numero variabile di operandi per ciascuna istruzione
- Formati di istruzione di lunghezza non uniforme



Modello Semplificato

È importante sottolineare che lo schema architetturale di riferimento è necessariamente semplificato per scopi didattici. Un microprocessore reale presenta una struttura molto più articolata, che include:

- **ALU** (Arithmetic Logic Unit) per operazioni logico-aritmetiche
- **FPU** (Floating Point Unit) dedicata ai calcoli in virgola mobile
- Unità specializzate per istruzioni complesse
- Unità di calcolo vettoriale per elaborazioni SIMD

Nonostante la semplificazione, il modello di riferimento mantiene la sua validità concettuale e permette di comprendere i principi fondamentali del funzionamento dei processori moderni.

Made with GAMMA

Architettura CISC: Microprogrammi e Interpretazione

01

Decode

Decodifica dell'istruzione ISA per comprenderne il tipo e la struttura

02

Operand Fetch

Recupero degli operandi necessari all'esecuzione dell'istruzione

03

Execute

Esecuzione del microprogramma specifico per completare l'operazione

Il Microprocessore a Interprete

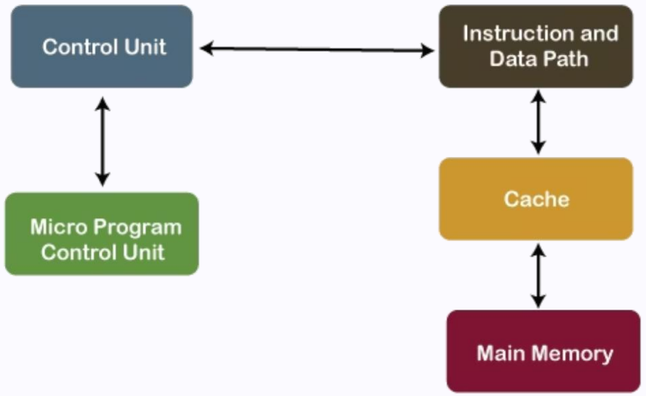
Il modello di processore CISC appena descritto rappresenta un esempio di **microprocessore "a interprete"**. Questa denominazione deriva dal fatto che il processore contiene al suo interno microprogrammi e microcodice che fungono da strato intermedio tra l'ISA e l'hardware effettivo.

Ogni singola istruzione dell'ISA attraversa un percorso complesso che richiede più fasi sequenziali:

1. L'istruzione viene prelevata dalla memoria
2. Viene decodificata per identificarne tipo e parametri
3. Vengono recuperati gli operandi richiesti
4. Viene avviato il microprogramma di esecuzione specifico

Data Path Multi-ciclo

Questa architettura implica che ogni singola istruzione dell'ISA abbia uno **svolgimento a più cicli**. Il data path, ovvero il percorso dei dati all'interno del processore, è scandito dal clock della CPU e richiede numerosi cicli per completare anche le operazioni più semplici.



CISC Architecture

Data Path: Il percorso che i dati seguono all'interno del processore durante l'esecuzione delle istruzioni. I suoi cicli sono sincronizzati con il clock della CPU.

Caratteristiche dell'Architettura CISC

Complex Instruction Set Code

Le architetture **CISC** (Complex Instruction Set Code) rappresentano un paradigma di progettazione dei processori caratterizzato da un approccio che privilegia la ricchezza e la complessità del set di istruzioni. Questo modello architetturale è stato predominante per decenni e continua ad avere un ruolo importante nell'informatica moderna.

Set di Istruzioni Numeroso

Le architetture CISC possiedono centinaia di istruzioni diverse, ciascuna progettata per eseguire operazioni specifiche e complesse, dalla manipolazione di stringhe alle operazioni aritmetiche avanzate.

Ampiezza Variabile

Le istruzioni CISC hanno lunghezze molto variabili, da pochi byte fino a diverse decine di byte, a seconda della complessità dell'operazione e del numero di operandi coinvolti.

Decode Complessa

La fase di decodifica richiede un data path articolato a più passi, necessario per interpretare correttamente la varietà di formati e modalità di indirizzamento disponibili.

Vantaggi delle Architetture CISC

Portabilità del Software

Una delle caratteristiche più preziose delle architetture CISC è la facilitazione della portabilità del software. L'insieme dei microprogrammi, che costituisce l'interprete del processore, può essere trasportato su processori più recenti mantenendo la compatibilità con il software esistente.

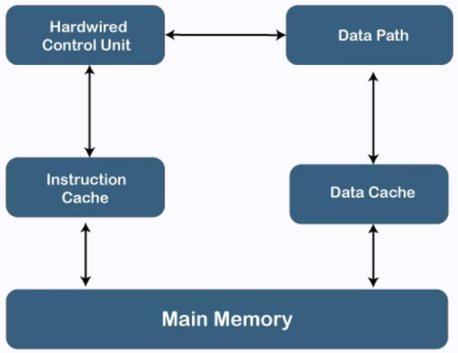
Questo aspetto è stato cruciale per il successo commerciale di architetture come x86, permettendo di mantenere la compatibilità con decenni di software legacy.

Programmabilità in Assembly

Le architetture CISC sono particolarmente **adatte per essere programmate direttamente in linguaggio Assembly**. La ricchezza del set di istruzioni permette ai programmatori di scrivere codice più compatto ed espressivo, con istruzioni di alto livello che mappano direttamente concetti algoritmici complessi.

Questo rende la programmazione a basso livello più produttiva e manutenibile rispetto ad architetture con set di istruzioni ridotti.

Architettura RISC: Semplicità ed Efficienza



RISC Architecture

Reduced Instruction Set Code

In netto contrasto con l'approccio CISC, l'architettura **RISC** (Reduced Instruction Set Code) adotta una filosofia radicalmente diversa: semplificare il set di istruzioni per massimizzare la velocità di esecuzione.

Il principio fondamentale è che un processore con istruzioni semplici e veloci possa superare in prestazioni un processore con istruzioni complesse ma lente.

1

Data Path Singolo

Ogni istruzione RISC possiede un data path a singolo passo, permettendo l'esecuzione in un numero ridotto di cicli di clock, tipicamente uno o due.

2

Set Limitato

Il set di istruzioni è volutamente **ridotto e ottimizzato**, contenendo solo le operazioni fondamentali necessarie per la computazione generale.

3

Lunghezza Costante

Tutte le istruzioni hanno **lunghezza fissa**, tipicamente 32 bit, semplificando eno

4

Operandi Fissi

Il numero di operandi per istruzione è **costante e predefinito**, eliminando la necessità di decodifiche complesse per determinare la struttura dell'istruzione.

Prestazioni e Trade-off

La fase di decode nelle architetture RISC è estremamente breve grazie alla semplicità e uniformità delle istruzioni. Non esistono microprogrammi da eseguire: ogni istruzione è implementata direttamente in hardware con circuiti dedicati, permettendo esecuzioni rapidissime.

Velocità di Esecuzione

Un'elaborazione RISC risulta **nettamente più veloce** rispetto a una equivalente CISC, con miglioramenti prestazionali di almeno un ordine di grandezza (10x). Un'istruzione CISC complessa, che richiede molti passi nel data path, equivale tipicamente a numerose istruzioni RISC con data path singolo.

Dimensione del Codice

Il rovescio della medaglia è che i **programmi RISC sono significativamente più lunghi** rispetto ai loro equivalenti CISC. Ciò che può essere espresso in una singola istruzione CISC richiede multiple istruzioni RISC, aumentando sia la dimensione del codice che l'utilizzo della memoria.

CRISC: L'Evoluzione Ibrida

Complex Reduced Instruction Set Code

L'analisi delle architetture CISC e RISC rivela che nessuna delle due architetture è ideale in tutti gli scenari. Ciascuna presenta vantaggi significativi ma anche limitazioni intrinseche che ne vincolano l'applicabilità e le prestazioni.

1

Sfide RISC

Maggiori difficoltà per la portabilità del software tra generazioni diverse di processori

2

Compilatori Complessi

I compilatori RISC devono essere molto sofisticati per ottimizzare il codice macchina

3

Assembly Difficile

Programmazione diretta in Assembly praticamente impossibile per mancanza di istruzioni di alto livello

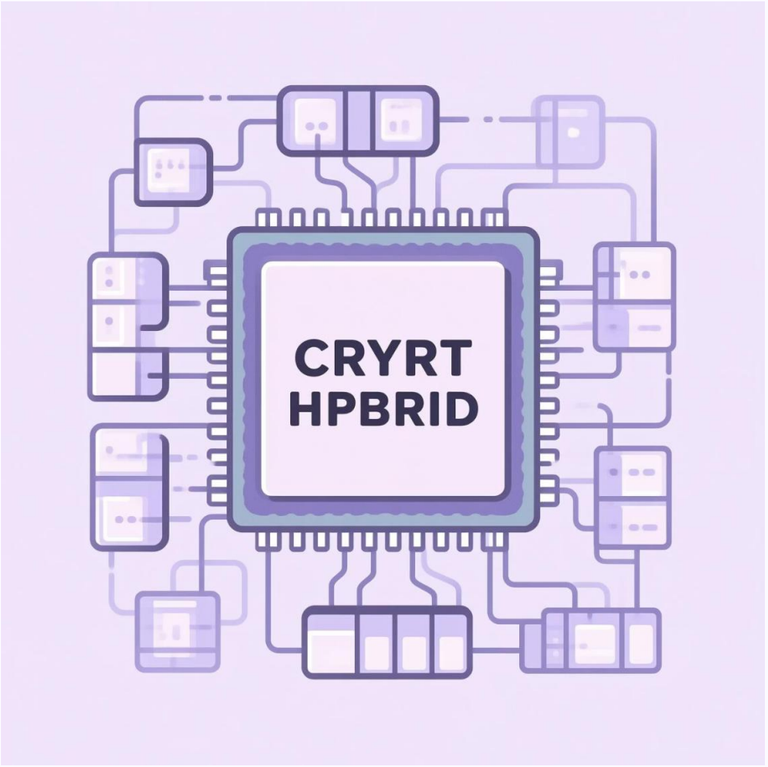
La Soluzione Ibrida CRISC

La tendenza attuale nell'industria dei microprocessori è l'implementazione di **architetture ibride CRISC** (Complex Reduced Instruction Set Code), che combinano i punti di forza di entrambi gli approcci.

Questi processori mantengono esteriormente un'interfaccia CISC, rispettando il modello di von Neumann e garantendo la compatibilità con il vasto ecosistema software esistente. Tuttavia, internamente, implementano sottosistemi basati su principi RISC.

Architettura a Due Livelli

- **Livello esterno CISC:** Mantiene la compatibilità con ISA complesse come x86, garantendo la portabilità del software
- **Livello interno RISC:** Le istruzioni CISC vengono tradotte in micro-operazioni RISC che vengono eseguite ad alta velocità
- **Esecuzione ibrida:** Le istruzioni semplici e comuni vengono eseguite direttamente in modalità RISC, mentre quelle
- complesse utilizzano microprogrammi



Processori moderni come Intel Core e AMD Ryzen sono esempi eccellenti di architetture CRISC, in grado di eseguire codice x86 legacy mantenendo prestazioni competitive con processori RISC puri.

Vantaggi dell'Approccio CRISC

L'architettura CRISC rappresenta il **meglio dei due mondi**:

- Compatibilità software delle architetture CISC
- Velocità di esecuzione delle architetture RISC
- Ottimizzazione per le istruzioni più frequenti
- Flessibilità per gestire operazioni complesse

Limiti Strutturali del Modello SISD/CISC

I Colli di Bottiglia dell'Architettura di von Neumann

Il modello SISD/CISC dell'architettura di von Neumann, nonostante sia stato la base fondamentale per decenni di evoluzione informatica, si scontra oggi con problemi strutturali intrinseci che ne limitano significativamente le prestazioni potenziali. Questi limiti non possono essere superati attraverso semplici ottimizzazioni, ma richiedono approcci architetturali fondamentalmente diversi.

Limite SISD: Esecuzione Sequenziale

Il principio **Single Instruction, Single Data** implica che una singola istruzione operi su un singolo dato nell'unità di tempo. Questo approccio strettamente sequenziale rappresenta un limite fondamentale e da tempo inaccettabile per le moderne esigenze computazionali.
Le applicazioni contemporanee, dal rendering grafico all'intelligenza artificiale, richiedono l'elaborazione simultanea di enormi quantità di dati. Un'architettura puramente SISD non può sfruttare questo parallelismo intrinseco, lasciando inutilizzata gran parte della potenza computazionale potenzialmente disponibile.

Limite CISC: Memory Wall

Il secondo problema critico è il cosiddetto **"memory wall"** o muro della memoria: il divario crescente tra la velocità del processore e quella della memoria. Mentre il clock della CPU opera nell'ordine dei GigaHertz (miliardi di cicli al secondo), il clock del bus di memoria rimane nell'ordine dei MegaHertz (milioni di cicli al secondo).
Questo disallineamento causa un **ritardo cronico** nell'accesso alla memoria. La fase di decode CISC, già complessa e onerosa, viene ulteriormente penalizzata da questo gap prestazionale. Il processore, anche se estremamente veloce, trascorre la maggior parte del tempo in attesa che istruzioni e dati arrivino dalla memoria.

Strategie di Mitigazione

Superare il Limite SISD

Molte delle soluzioni moderne per incrementare le prestazioni tendono a **parallelizzare l'esecuzione**:

- Architetture SIMD (Single Instruction, Multiple Data)
- Multi-core e multi-threading
- GPU computing per elaborazioni massivamente parallele
- Pipeline di esecuzione profonde
- Esecuzione fuori ordine (out-of-order execution)

Superare il Limite CISC

Per affrontare il memory wall, i moderni processori implementano **gerarchie di memoria complesse**:

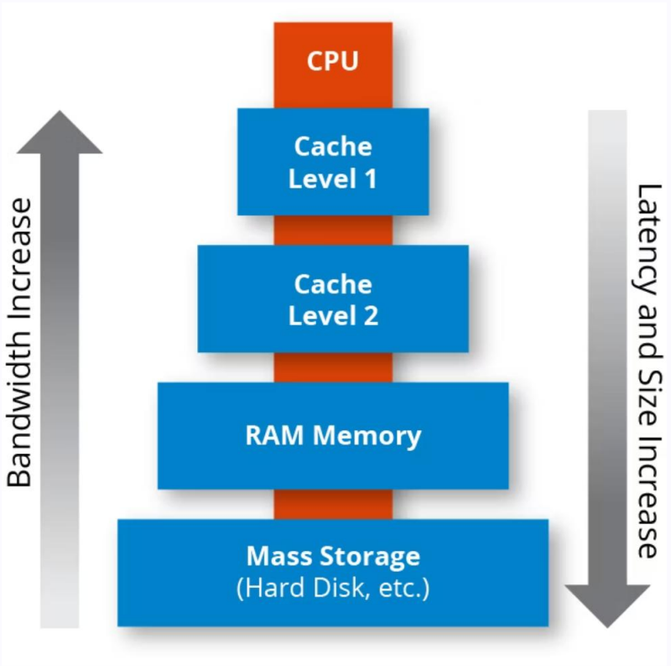
- Memorie cache multiple a più livelli
- Prefetching intelligente dei dati
- Buffer di predizione dei branch
- Registri interni estesi
- Architetture CRISC per istruzioni più efficienti

L'obiettivo è fornire istruzioni e operandi dalla memoria alla stessa velocità che il processore impiega per eseguirle.

La Memoria Cache: Soluzione al Memory Wall

Cache Memory: Il Tampone Intelligente

Un modo particolarmente ingegnoso per diminuire drasticamente gli accessi al bus e alla memoria principale, contrastando efficacemente i limiti dell'architettura CISC, consiste nel dotare il calcolatore di una **memoria tampone** ad alta velocità posizionata strategicamente tra il processore e il bus di sistema.
Questa memoria speciale è chiamata Cache Memory e rappresenta uno degli elementi architetturali più importanti nei processori moderni. La sua presenza permette di ridurre significativamente il tempo medio di accesso ai dati, mascherando la latenza della memoria principale.



Principio di Funzionamento

La cache sfrutta il **principio della località**, sia spaziale che temporale, caratteristica intrinseca della maggior parte dei programmi:

- **Località temporale:** I dati recentemente acceduti hanno alta probabilità di essere riacceduti a breve
- **Località spaziale:** I dati vicini in memoria a quelli appena acceduti hanno alta probabilità
- di essere acceduti a breve

1. **Richiesta CPU**
   Il processore richiede un dato a un determinato indirizzo di memoria
2. **Verifica Cache**
   Il sistema controlla se il dato è presente nella cache (operazione velocissima)
3. **Hit o Miss**
   Se presente (hit) viene fornito immediatamente; se assente (miss) si accede alla RAM
4. **Aggiornamento**
   In caso di miss, viene caricata una linea di cache contenente il dato e dati adiacenti

Tecnologia e Prestazioni

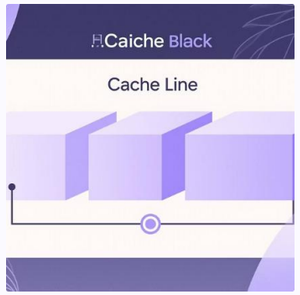
Nota tecnica: La memoria cache è realizzata in tecnologia **SRAM** (Static Random Access Memory), che è molto più veloce della **DRAM** (Dynamic RAM) utilizzata per la memoria principale, ma anche più costosa e meno densa.

La differenza di velocità è drammatica: mentre l'accesso alla DRAM richiede decine o centinaia di nanosecondi, l'accesso alla SRAM cache può completarsi in pochi nanosecondi, allineandosi meglio con la velocità del processore.

Linee di Cache e Località Spazio-Temporale

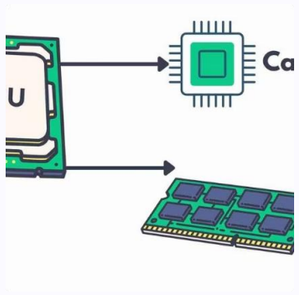
Il Meccanismo delle Linee di Cache

Quando il processore legge dalla memoria un dato situato a un determinato indirizzo, il sistema di cache non si limita a trasferire solamente quel singolo valore. Invece, **un intero blocco di memoria** contenente l'indirizzo richiesto viene copiato nella cache. Questi blocchi predefiniti sono chiamati linee di cache (cache lines).



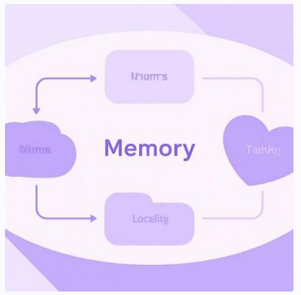
Struttura della Linea

Una tipica linea di cache contiene 64 byte di dati contigui dalla memoria principale, insieme a metadati come tag e bit di validità



Caricamento Proattivo

Quando viene richiesto un byte, l'intero blocco circostante viene portato in cache nello stesso tempo di bus



Sfruttamento Località

Le istruzioni successive e i dati correlati hanno alta probabilità di trovarsi già in cache

Il Principio della Località Spazio-Temporale

La strategia delle linee di cache si basa su un principio fondamentale del comportamento dei programmi: il **principio della località spazio-temporale**. Questa osservazione empirica, confermata da decenni di analisi del comportamento del software, afferma che un programma utilizza, entro un breve intervallo di tempo, solo una piccola parte del suo spazio degli indirizzi totale.

Località Temporale

Se un programma accede a una locazione di memoria, è molto probabile che acceda nuovamente alla stessa locazione nel prossimo futuro. Questo spiega perché:

- I cicli di programma riutilizzano le stesse variabili ripetutamente
- Le funzioni frequentemente chiamate rimangono "calde" in cache
- Le strutture dati attive rimangono accessibili rapidamente

La cache mantiene questi dati "caldi" immediatamente disponibili, evitando costosi accessi alla memoria principale.

Località Spaziale

Se un programma accede a una locazione di memoria, è molto probabile che acceda a locazioni vicine nel prossimo futuro. Gli indirizzi utilizzati sono numericamente prossimi tra loro perché:

- Le istruzioni di un programma sono memorizzate sequenzialmente
- Gli array e le strutture dati occupano regioni contigue
- Lo stack cresce in modo sequenziale

Caricare interi blocchi di memoria sfrutta questa prevedibilità, minimizzando i miss futuri.

Vantaggi Prestazionali

Grazie alla memoria cache realizzata in SRAM, **molto più veloce** della memoria DRAM principale, il processore può ottenere nell'immediato futuro i valori desiderati (per esempio, la prossima istruzione da eseguire o l'elemento successivo di un array) senza dover accedere al bus di sistema, che opera a velocità molto inferiori rispetto al clock della CPU.

Gestione della Cache: Hit, Miss e Replacement

Il Ciclo di Vita di un Accesso alla Memoria

1. Richiesta di Lettura
Il processore genera una richiesta di lettura per un determinato indirizzo di memoria

2. Ricerca in Cache
Il sistema verifica prima di tutto se l'informazione richiesta è presente nella cache memory

3. Cache Hit
Se il dato è presente (hit), viene fornito immediatamente alla CPU senza accesso al bus

4. Cache Miss
Se il dato non è presente (miss), si deve accedere alla memoria principale tramite bus

5. Caricamento e Replacement
Si carica una nuova linea di cache contenente il dato, sostituendo la linea meno usata di recente (LRU)

Cache Hit: Il Caso Ottimale

Quando si verifica un **cache hit**, il sistema raggiunge la sua massima efficienza. Il dato richiesto viene trovato nella cache e fornito al processore in pochi cicli di clock, tipicamente 1-4 cicli per la cache L1.

Un alto tasso di hit (hit rate) è fondamentale per le prestazioni:

- Hit rate del 95-99% sono comuni in applicazioni ben ottimizzate
- Ogni punto percentuale di miglioramento può tradursi in significativi guadagni prestazionali
- La località del codice e dei dati influenza direttamente il tasso di hit

Cache Miss: Il Penalty Inevitabile

Un **cache miss** comporta un significativo penalty prestazionale. Il processore deve:

1. Accedere al livello successivo della gerarchia di memoria (L2, L3, o RAM)
2. Attendere decine o centinaia di cicli di clock per il dato
3. Caricare l'intero blocco (linea di cache) dalla memoria
4. Potenzialmente espellere una linea esistente dalla cache

Miss Penalty: L'accesso alla RAM può richiedere 100-300 cicli di clock, contro i 1-4 cicli della cache L1. Questo rapporto spiega l'importanza cruciale dell'ottimizzazione della cache.

Politica di Sostituzione: LRU

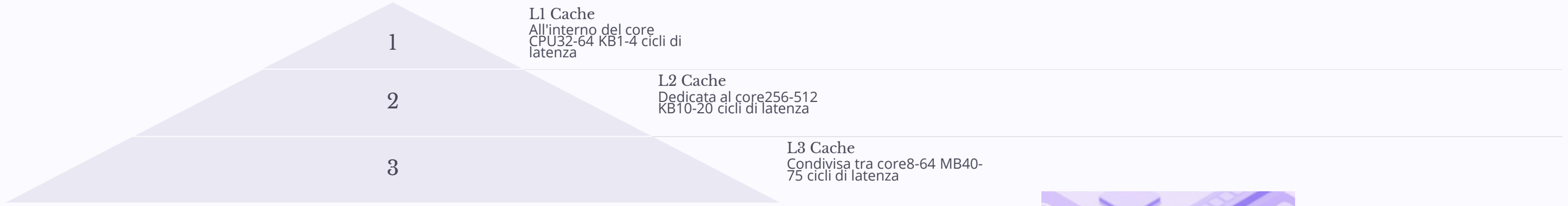
Quando una nuova linea di cache deve essere caricata ma la cache è piena, il sistema deve decidere quale linea esistente espellere. La strategia più comune è **LRU (Least Recently Used)**: viene sovrascritta la linea di cache che è stata utilizzata meno recentemente.

Questa politica si basa sul principio di località temporale: se una linea non è stata acceduta per molto tempo, probabilmente non sarà necessaria nel prossimo futuro. Alternative includono algoritmi pseudo-LRU, random replacement, e politiche più sofisticate basate su predizione.

Gerarchia delle Cache nei Calcolatori Moderni

Architettura Multi-Livello

I calcolatori moderni implementano una **gerarchia complessa di memorie cache** organizzate su più livelli, ciascuno con caratteristiche specifiche di dimensione, velocità e prossimità al processore. Questa struttura a più strati bilancia sapientemente velocità, capacità e costi, creando un sistema di memoria che avvicina le prestazioni della SRAM veloce alla capacità della DRAM economica.

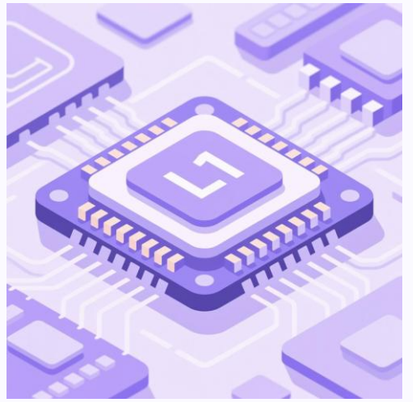


Livello 1 (L1): Velocità Massima

La cache di **Livello 1** è integrata direttamente all'interno del processore, nel cuore stesso di ciascun core di elaborazione. Si tratta della memoria più veloce disponibile nel sistema, con latenze di accesso di appena 1-4 cicli di clock.

Caratteristiche della L1:

- Split cache: Divisa in I-cache (istruzioni) e D-cache (dati)
- Dimensione tipica: 32-64 KB per tipo
- Velocità identica al clock del processore
- Associatività elevata (8-way o superiore)



Livello 2 (L2): Capacità Intermedia

La cache di **Livello 2** è collegata direttamente al processore, tipicamente dedicata a ciascun core in architetture multi-core moderne. Rappresenta un compromesso tra velocità e capacità, offrendo uno storage intermedio più capiente della L1 ma ancora molto veloce rispetto alla memoria principale.

Capacità	Latenza	Posizionamento
256-512 KB per core, unified (istruzioni e dati insieme)	10-20 cicli di clock, ancora molto veloce ma significativamente più lenta della L	Dedicata al singolo core, ottimizzata per ridurre conflitti tra thread

Livello 3 (L3): Cache Condivisa

La cache di **Livello 3** è montata sulla scheda madre o integrata nel package del processore, ma condivisa tra tutti i core del processore. Rappresenta l'ultimo livello di cache prima della memoria principale.

Funzioni della L3:

- **Buffer unificante:** Riduce il traffico verso la memoria principale
- **Coerenza tra core:** Facilita la condivisione dei dati tra core diversi
- **Victim cache:** Accoglie linee espulse dalle cache L2 dei vari core
- **Grande capacità:** 8-64 MB, a volte anche oltre nei processori server

La latenza della L3 è di 40-75 cicli, significativamente più alta rispetto ai livelli superiori, ma ancora molto inferiore ai 200-300 cicli necessari per accedere alla RAM.

Cache Inclusiva vs Esclusiva: La L3 può essere *inclusiva* (contiene copie di tutto ciò che è in L1/L2) o *esclusiva* (contiene solo ciò che non è in L1/L2), con differenti trade-off prestazionali.

Impatto Sulle Prestazioni Complessive

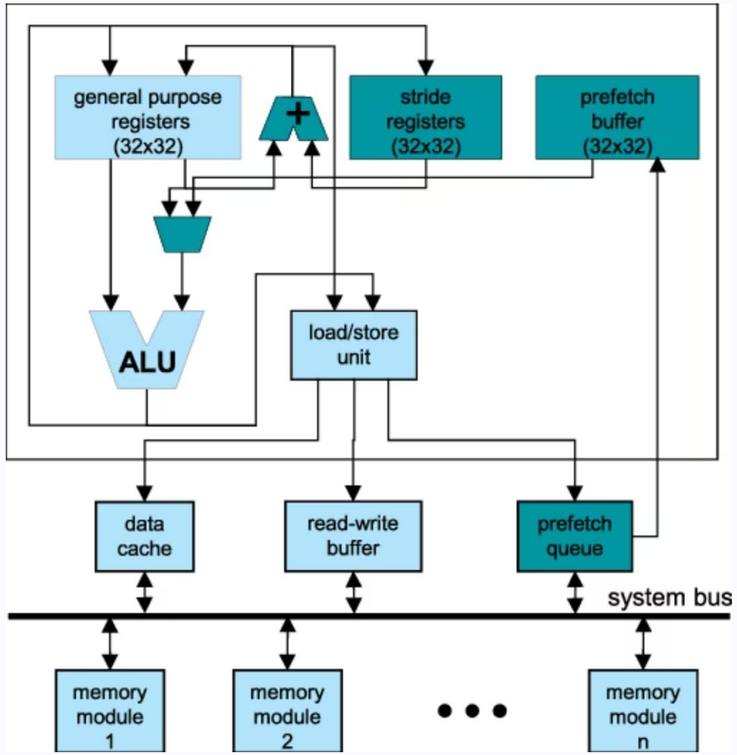
Questa **gerarchia a tre livelli** permette ai processori moderni di mantenere prestazioni elevate nonostante il divario crescente tra velocità della CPU e velocità della memoria. Un sistema ben progettato può raggiungere un tempo medio di accesso alla memoria di pochi cicli, mascherando quasi completamente la latenza della DRAM attraverso un'efficace gestione della gerarchia di cache.

Prefetch: Il Primo Passo verso il Parallelismo

La Rivoluzione del Buffer Interno

Un modo per aumentare il parallelismo d'esecuzione, e quindi di superare i limiti di un'architettura SISD (Single Instruction Single Data), fu quello di caricare nel processore più istruzioni oltre a quella richiesta. Questa tecnica rappresentò uno dei primi tentativi di ottimizzazione delle prestazioni dei processori, introducendo il concetto di anticipazione nell'accesso alla memoria.

Fin dagli esordi dell'informatica moderna, i progettisti si resero conto che il processore passava molto tempo in attesa dei dati provenienti dalla memoria. Per risolvere questo problema, i processori furono dotati di una **coda di Prefetch**, ovvero di un buffer interno in cui il processore memorizzava i successivi 6 o 8 byte consecutivi a quello appena letto dalla memoria.



Vantaggio Principale

Con un solo accesso alla memoria, si aveva a disposizione una serie di valori che potevano essere usati successivamente come istruzioni od operandi

Riduzione degli Accessi

Eliminazione della necessità di accedere ripetutamente al bus di sistema per ogni singolo byte

Efficienza Migliorata

Sfruttamento della località spaziale: le istruzioni consecutive tendono ad essere utilizzate in sequenza

Made with GAMMA

Pipeline: La Catena di Montaggio del Processore

Ben presto, alla coda di **Prefetch**, fu affiancato un sistema a **Pipeline** che ha lo scopo di sfruttare il concetto di catena di montaggio applicato all'elaborazione delle istruzioni. Questa tecnica rivoluzionò completamente il modo in cui i processori gestivano l'esecuzione del codice, introducendo un parallelismo a livello di istruzione che avrebbe caratterizzato tutte le architetture moderne.

Il Principio della Pipeline

Invece di eseguire un'istruzione completamente e solo al termine elaborare la sua successiva, si può avviare l'istruzione subito dopo che la precedente è stata inserita nel data path. Questo approccio permette di sovrapporre temporalmente l'esecuzione di più istruzioni, aumentando drasticamente il throughput del processore.

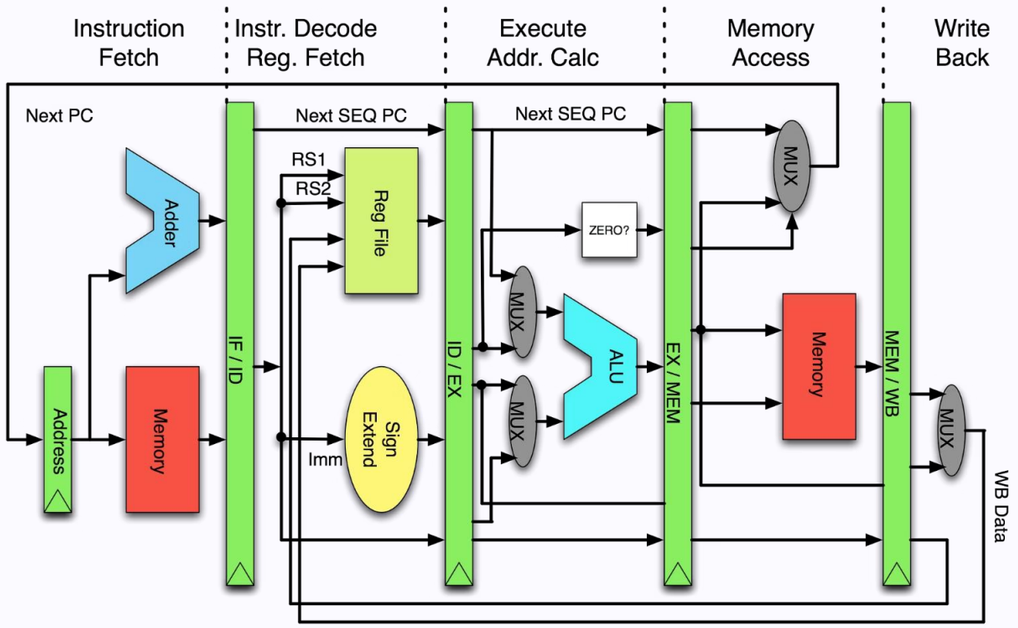
Per esempio, basta che la prima istruzione si trovi in fase di **decode**, e la successiva può essere posta in stato di **fetch**. Così come in una catena di montaggio industriale, un nuovo pezzo può essere lavorato anche se il precedente non è ancora stato completato: basta che le fasi, dette anche **stadi della pipeline**, non si sovrappongano o interferiscano tra loro.

1. **Fetch**
   Preleva istruzione
2. **Decode**
   Decodifica operazione
3. **Execute**
   Esegue calcolo
4. **Memory**
   Accede memoria
5. **Write Back**
   Scrive risultato

Analogia Industriale

Immaginate una fabbrica di automobili: mentre un'auto viene verniciata, un'altra può essere assemblata, e una terza può avere il motore installato. Ogni stazione lavora contemporaneamente su auto diverse, aumentando la produzione totale.

I Cinque Stadi della Pipeline Classica



L'elaborazione di un'istruzione da parte di un processore si compone di cinque passaggi fondamentali, ognuno dei quali rappresenta uno stadio distinto della pipeline. Questa suddivisione permette di ottimizzare l'utilizzo delle risorse hardware, assicurando che ogni componente del processore sia sempre attivo.

01

IF - Instruction Fetch

Lettura dell'istruzione dalla memoria principale. Il program counter indica l'indirizzo dell'istruzione da prelevare, che viene caricata nel registro istruzioni del processore.

02

ID - Instruction Decode

Decodifica dell'istruzione e lettura degli operandi dai registri. L'unità di controllo interpreta l'opcode e identifica quali registri contengono i dati necessari per l'operazione.

03

EX - Execution

Esecuzione vera e propria dell'istruzione tramite l'ALU (Arithmetic Logic Unit). Vengono eseguite operazioni aritmetiche, logiche o di calcolo degli indirizzi.

04

MEM - Memory Access

Attivazione della memoria per operazioni di load/store. Questo stadio è utilizzato solo per certe istruzioni che richiedono accesso alla memoria dati.

05

WB - Write Back

Scrittura del risultato nel registro di destinazione opportuno. Il valore calcolato viene memorizzato nel file dei registri per utilizzi futuri.

Pipeline e il Collo di Bottiglia di von Neumann



Superare il Collo di Bottiglia

La **Pipeline** sopperisce così alle attese di CPU veloci nei confronti di memorie lente, il famoso *collo di bottiglia di von Neumann*. Questo problema, identificato fin dagli albori dell'informatica, deriva dal fatto che la CPU e la memoria condividono lo stesso bus per trasferire sia istruzioni che dati, creando un limite fisico alla velocità di elaborazione.

Una volta dotato di **Pipeline**, in un processore si è notato che lo stadio di esecuzione (EX) è tipicamente il più lento e complesso: lo stadio precedente (ID) può fornire istruzioni decodificate più velocemente di quanto lo stadio di esecuzione, implementato nell'ALU, possa elaborarle. Questo sbilanciamento può creare un nuovo collo di bottiglia interno alla pipeline stessa.

Il Problema Fondamentale

Così, una **Pipeline** a 5 stadi trasporta contemporaneamente cinque istruzioni in catena di montaggio, con ciascuna istruzione in uno stadio diverso di elaborazione. Questo significa che, idealmente, ogni ciclo di clock può completare un'istruzione, nonostante ogni singola istruzione richieda cinque cicli per essere completamente processata.

1. **Problema Identificato**
   CPU veloci attendono memorie lente
2. **Soluzione Pipeline**
   Sovrapposizione temporale delle istruzioni
3. **Nuovo Collo**
   Lo stadio EX diventa il più lento
4. **Evoluzione**
   Necessità di più ALU parallele

Ecco allora che sui processori moderni sono montate più ALU in parallelo, in modo da servire velocemente ogni istruzione che arriva allo stadio di execute, evitando che questo diventi un ulteriore punto di congestione.

Superscalarità: Moltiplicare la Potenza

Architettura Superscalare

Ecco allora che sui processori sono montate più ALU (Arithmetic Logic Unit) in modo da servire velocemente ogni istruzione che arriva allo stadio di execute. In questo caso il processore è detto **Superscalare**, termine che indica la capacità di iniziare l'esecuzione di più di un'istruzione durante lo stesso ciclo di clock.

In questo modo è possibile dotare i processori anche di due, quattro o più pipeline differenti che operano in parallelo. Ogni pipeline può elaborare un flusso di istruzioni indipendente, moltiplicando effettivamente il throughput del processore. I processori superscalari moderni possono avere fino a 6-8 pipeline parallele, permettendo l'esecuzione simultanea di molteplici operazioni.

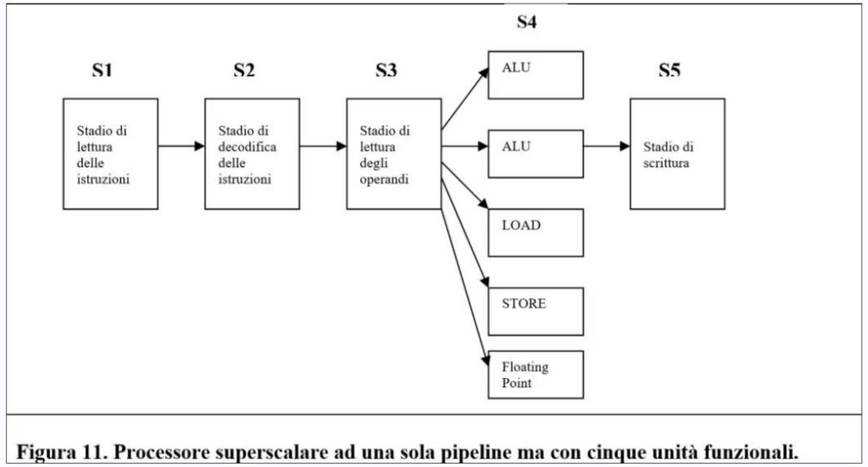
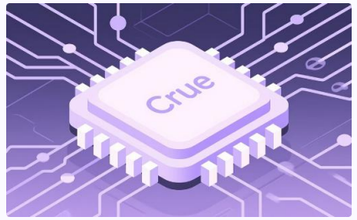
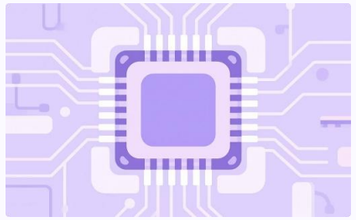


Figura 11. Processore superscalare ad una sola pipeline ma con cinque unità funzionali.



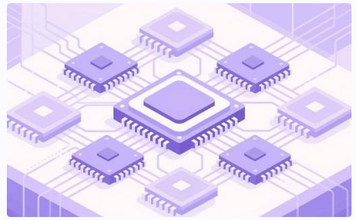
Processore Scalare

Una singola ALU, una pipeline. Esecuzione sequenziale delle istruzioni con throughput limitato.



Processore Superscalare

Multiple ALU, multiple pipeline. Esecuzione parallela con throughput moltiplicato e maggiore efficienza.



Architetture Moderne

Decine di unità di esecuzione specializzate che lavorano simultaneamente su diversi tipi di operazioni.

L'architettura superscalare rappresenta un fondamentale passo avanti nell'evoluzione dei processori, permettendo di sfruttare il parallelismo a livello di istruzione (ILP - Instruction Level Parallelism) senza richiedere modifiche ai programmi esistenti.

I Problemi della Pipeline: Ostacoli al Parallelismo

Tutte queste metodologie di ottimizzazione, però, vengono vanificate o significativamente limitate da due situazioni critiche che si presentano frequentemente durante l'esecuzione del codice. Questi problemi rappresentano le sfide principali che i progettisti di processori devono affrontare per mantenere l'efficienza della pipeline.

1. Istruzioni di Salto (Branch Instructions)

Nel primo caso la pipeline viene del tutto persa se l'istruzione corrente è un salto (**branch**) distante dall'istruzione successiva che è attualmente in pipeline. Quando si verifica un salto, tutte le istruzioni che erano state precaricate nelle pipeline devono essere scartate perché appartengono a un flusso di esecuzione che non verrà seguito.
In questo caso viene persa anche la **cache**, poiché le istruzioni e i dati che erano stati prefetchati non sono più rilevanti per il nuovo percorso di esecuzione. Questo fenomeno è noto come *pipeline flush* o svuotamento della pipeline, e può causare una perdita temporanea di prestazioni significativa, equivalente a diversi cicli di clock sprecati.

2. Dipendenza dei Dati (Data Dependencies)

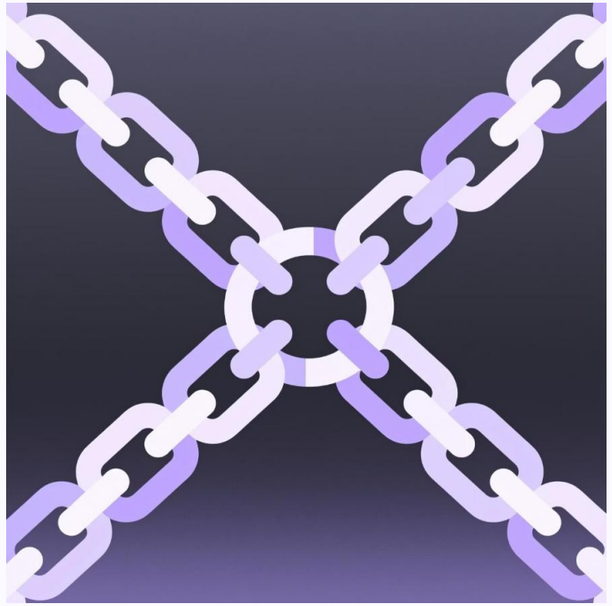
Nel secondo caso una pipeline deve essere interrotta se l'istruzione successiva necessita, come operando, del risultato finale dell'istruzione precedente. Questo tipo di dipendenza è chiamato **data hazard** o *RAW hazard* (Read After Write).
La pipeline non può procedere perché l'istruzione dipendente non può essere eseguita finché non ha a disposizione il valore corretto prodotto dall'istruzione precedente. Questo causa uno *stall* (arresto temporaneo) della pipeline, con conseguente perdita di efficienza e riduzione del throughput del processore.



⚠ Impatto sulle Prestazioni

Questi problemi possono ridurre l'efficienza della pipeline dal 100% teorico fino al 30-40% in casi reali, rendendo fondamentali le tecniche di ottimizzazione avanzate.

Esempio di Dipendenza dei Dati



Codice di Esempio

```
Istruzione 1: A = B + C;
Istruzione 2: D = A + 1;
```

In questo caso, l'**Istruzione 2** ha una dipendenza diretta dall'**Istruzione 1** perché utilizza il valore di **A** come operando di input.

Il Problema RAW

Consideriamo una situazione molto comune nella programmazione che illustra perfettamente il problema delle dipendenze dei dati e come queste influenzano negativamente l'esecuzione della pipeline.

1. **Ciclo 1-5: Istruzione 1**
   L'Istruzione 1 entra in pipeline ed esegue: Fetch → Decode → Execute (B+C) → Memory → Write Back (scrive A)
2. **Ciclo 2: Istruzione 2 si blocca**
   L'Istruzione 2 entra in Fetch, ma quando arriva a Decode/Operand Fetch scopre che necessita del valore di A
3. **Stall della Pipeline**
   La pipeline che sta servendo l'Istruzione 2 deve interrompersi (**stall**) allo stadio di operand fetch, dato che A non è disponibile finché l'Istruzione 1 non ha completato lo stadio di Write Back
4. **Ripresa dell'Esecuzione**
   Solo dopo che A è stato scritto nel register file, l'Istruzione 2 può riprendere e completare la sua esecuzione

---

Questo tipo di dipendenza è estremamente comune nel codice reale e può causare numerosi stall della pipeline. Per mitigare questo problema, i processori moderni implementano tecniche come il **data forwarding** (o *bypassing*), che permette di inoltrare il risultato di un'istruzione direttamente allo stadio di esecuzione della successiva, senza aspettare che venga scritto nei registri.

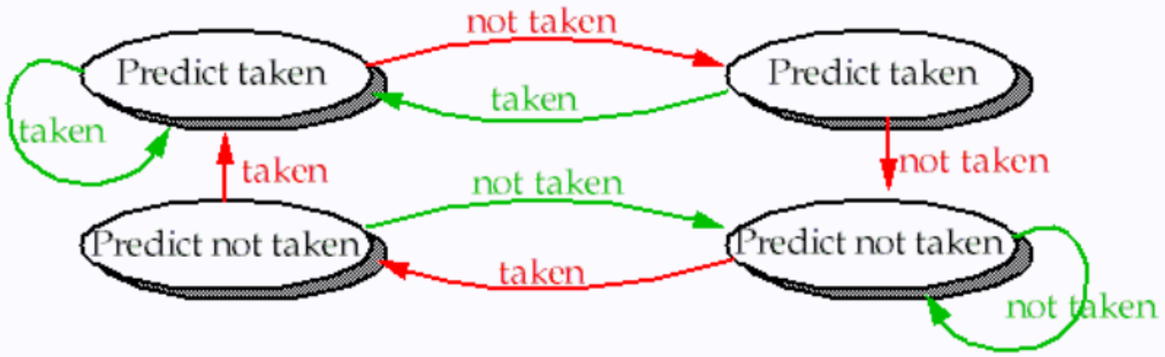
Esecuzione Predicativa: Prevedere il Futuro

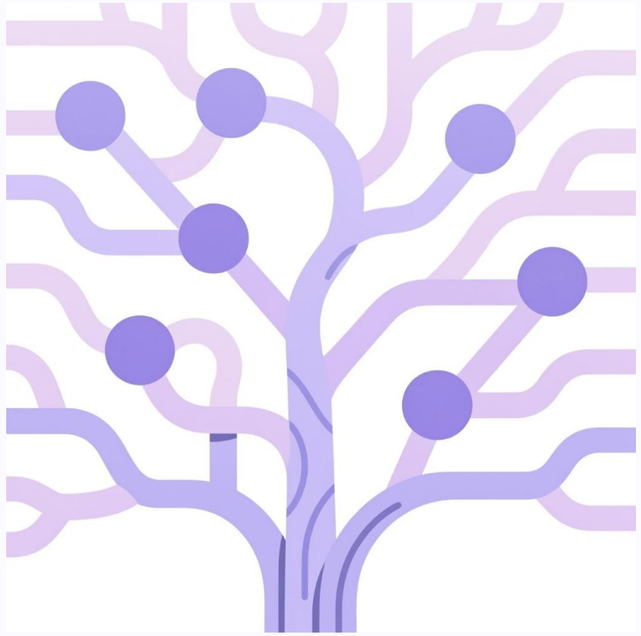
L'esecuzione predicativa, implementata in moduli del processore denominati *unità di previsione dei salti* (**Dynamic Branch Prediction**), cerca di prevenire la perdita delle pipeline a causa delle istruzioni di salto. Questa tecnica rappresenta uno dei più importanti avanzamenti nell'architettura dei processori moderni, permettendo di mantenere alta l'efficienza della pipeline anche in presenza di flussi di controllo complessi.

Come Funziona la Predizione

In questo caso le unità di predizione cercano, con vari algoritmi sofisticati che usano tabelle simili a memorie cache (chiamate **Branch Target Buffer** e **Pattern History Table**), di capire se un'istruzione di salto avverrà o meno. È importante sottolineare che questa previsione è del tutto non deterministica ma solo statistica o «storica», dato che il comportamento del salto viene analizzato e previsto basandosi sui pattern osservati durante l'esecuzione precedente del programma.

Per esempio, alcuni criteri considerano sempre «presi» (*taken*) i salti all'indietro, tipici dei cicli iterativi. In un ciclo for o while, il salto all'indietro avviene molto più spesso (ad ogni iterazione) rispetto all'uscita dal ciclo (una sola volta), rendendo statisticamente conveniente predire che il salto sarà eseguito.





Efficacia della Predizione

I predittori moderni raggiungono accuratezze del 95-98% nella previsione dei salti, riducendo drasticamente le penalità dovute ai branch misprediction.

Predittori Statici

Usano regole fisse (es: sempre preso, mai preso, backward taken)

Predittori Dinamici

Si adattano al comportamento del programma usando contatori e pattern storici

Predittori Neurali

Usano algoritmi avanzati simili alle reti neurali per predizioni complesse

Esecuzione Speculativa e Out-of-Order Execution

Il Rischio della Speculazione

Il **problema** di questa tecnica predicativa, che in realtà è molto efficiente nella maggior parte dei casi, si ha quando la previsione è sbagliata: le istruzioni eseguite inutilmente devono essere completamente scartate e lo stato della macchina deve essere ripristinato alla condizione precedente al salto. Questo processo di *rollback* richiede tempo e risorse, causando una penalità in termini di prestazioni.

L'esecuzione predicativa è anche nota come **esecuzione speculativa**, intendendosi quella elaborazione che computa anche il codice che potrebbe non essere mai effettivamente utilizzato nel flusso reale del programma. Il processore «scommette» su quale sarà il percorso di esecuzione e procede ottimisticamente lungo quella strada.

Esecuzione Fuori Ordine

Con l'**esecuzione fuori ordine** (*out-of-order execution*) si cerca di prevenire lo svuotamento delle pipeline a causa di dipendenze tra le istruzioni. Questa tecnica permette al processore di riordinare dinamicamente l'esecuzione delle istruzioni per massimizzare l'utilizzo delle risorse hardware.



Rilevamento Dipendenza

Il processore individua una dipendenza di dati in una pipeline e identifica lo stall imminente

Ricerca Istruzioni Indipendenti

Invece di bloccare completamente la pipeline, il processore cerca istruzioni «future» che non dipendono dal dato mancante

Esecuzione Anticipata

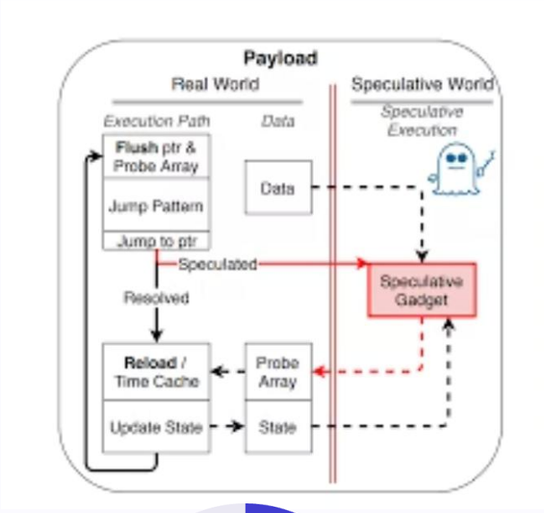
Le istruzioni indipendenti vengono eseguite in anticipo, anche se nella sequenza originale del programma dovevano essere eseguite dopo

Conservazione Pipeline

La pipeline viene quasi del tutto conservata: solo uno stadio rimane temporaneamente bloccato in attesa dell'operando mancente

In questo modo, una volta risolta la dipendenza originale, saranno già state eseguite altre istruzioni (quelle che avevamo chiamato istruzioni «future»), con conseguente significativa risalita delle prestazioni complessive del sistema. Questa tecnica richiede hardware complesso per tenere traccia delle dipendenze e riordinare i risultati, ma è fondamentale per i processori moderni ad alte prestazioni.

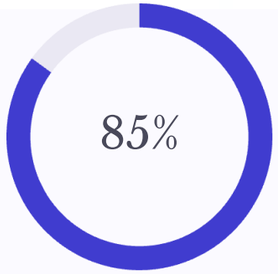
Out-of-Order Execution: Massimizzare l'Efficienza



Saltare le Dipendenze

Quando il processore individua una dipendenza in una pipeline, invece di «buttarla» completamente e attendere passivamente l'operando mancante causando uno stall totale, la salta temporaneamente e prosegue con istruzioni «future» che, secondo l'ordine del programma originale, dovrebbero essere eseguite solo dopo quella interrotta.

Questo approccio rivoluzionario permette di mantenere tutte le unità di esecuzione costantemente attive, minimizzando i tempi morti e massimizzando il throughput del processore.



Utilizzo Pipeline

Percentuale media di utilizzo della pipeline con esecuzione out-of-order rispetto al 40-50% con esecuzione in ordine



Miglioramento ILP

Fattore di incremento del parallelismo a livello di istruzione ottenibile con tecniche speculative avanzate



Istruzioni in Volo

Numero di istruzioni che processori moderni possono tracciare simultaneamente in vari stadi di esecuzione

Vantaggi Chiave

- La pipeline viene quasi del tutto conservata durante gli stall
- Un solo stadio rimane bloccato fino all'arrivo dell'operando mancante
- Tutte le altre unità di esecuzione continuano a lavorare su istruzioni indipendenti
- Una volta risolta la dipendenza, molte istruzioni future sono già state completate

Complessità Implementativa

- Richiede **Reorder Buffer** (ROB) per tracciare l'ordine originale delle istruzioni
- Necessita di **Reservation Stations** per memorizzare istruzioni in attesa di operandi
- Implementa **Register Renaming** per eliminare dipendenze artificiali (WAR, WAW)
- Utilizza complessi algoritmi di scheduling per scegliere quali istruzioni eseguire

Risultato Finale

Nonostante la complessità hardware richiesta, l'esecuzione out-of-order combinata con la predizione dei salti e l'esecuzione speculativa rappresenta il cuore delle architetture superscalari moderne, permettendo di raggiungere prestazioni straordinarie mantenendo la compatibilità con il codice esistente. Queste tecniche trasformano processori sequenziali in potenti motori di calcolo parallelo.

Condizioni Critiche nell'Esecuzione Fuori Ordine

L'esecuzione fuori ordine (out-of-order execution) rappresenta una delle tecniche più sofisticate nell'architettura dei processori moderni, ma introduce sfide significative nella gestione dello stato del sistema. Questa tecnica permette al processore di massimizzare l'utilizzo delle sue unità funzionali eseguendo istruzioni in un ordine diverso da quello del programma originale, ma richiede meccanismi complessi per garantire la correttezza dei risultati.



Gestione delle Dipendenze

Le istruzioni "future" possono essere eseguite anticipatamente solo se non presentano dipendenze con istruzioni attualmente in corso di esecuzione. Il processore deve analizzare continuamente le relazioni tra istruzioni per identificare quali possono essere eseguite in parallelo senza violare la semantica del programma.



Ripristino dell'Ordine

Non appena le istruzioni con dipendenze terminano la loro esecuzione, il processore riprende l'esecuzione in ordine (in-order execution). Questo garantisce che i risultati visibili al programma siano sempre coerenti con un'esecuzione sequenziale, mantenendo la correttezza semantica del codice.



Problema degli Interrupt

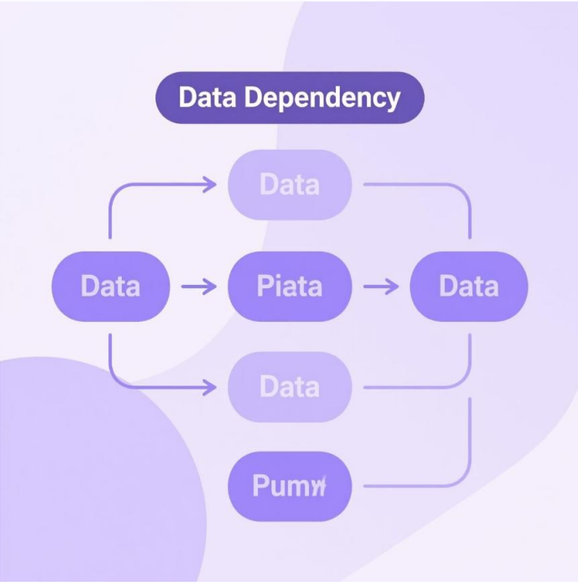
La condizione più critica si verifica quando il processore deve essere interrotto a causa di un interrupt. Se il processore si trova in una fase di esecuzione fuori ordine, lo stato del sistema potrebbe risultare incoerente, con alcune istruzioni parzialmente completate e altre già terminate.

> Per gestire gli interrupt durante l'esecuzione fuori ordine, il processore deve implementare meccanismi sofisticati di ripristino dello stato. Questo processo, noto come **precise exception handling**, richiede che il processore sia in grado di ritirare «in ordine» tutte le istruzioni eseguite fuori ordine, ricostruendo lo stato esatto della CPU come se l'esecuzione fosse avvenuta sequenzialmente fino al punto dell'interrupt.

Questo meccanismo di rollback è fondamentale per garantire la **correctness** del sistema e permette al sistema operativo di gestire interruzioni hardware, eccezioni e context switch mantenendo un modello di programmazione consistente e predicibile per il software.

Made with GAMMA

Dipendenze RAW: Read After Write



Gran parte del lavoro computazionale delle unità che gestiscono l'esecuzione fuori ordine è dedicato all'individuazione e alla risoluzione delle dipendenze tra istruzioni. Queste dipendenze determinano quali istruzioni possono essere eseguite in parallelo e quali devono attendere il completamento di operazioni precedenti.

Dipendenza RAW (Read After Write)

La dipendenza più classica e problematica è la **RAW (Read After Write)**, detta anche dipendenza "vera" o "di flusso". Si verifica quando un'istruzione deve leggere un dato che viene scritto da un'istruzione precedente. Questa è l'unica dipendenza che rappresenta un vincolo reale nell'ordine di esecuzione delle istruzioni.

```
Istruzione1: A = B + C
Istruzione2: D = A + 1
```

L'Istruzione2 contiene una dipendenza RAW su A: non può essere eseguita prima che l'Istruzione1 abbia completato la scrittura del registro A. Violare questo ordine produrrebbe un risultato scorretto.

Altre Tipologie di Dipendenze

Oltre alla RAW, esistono altre due categorie di dipendenze che, pur non essendo dipendenze "vere", devono essere gestite:

- **WAR (Write After Read)**: un'istruzione scrive un registro che un'istruzione precedente deve ancora leggere. Questa è una dipendenza "anti" che può essere risolta con tecniche come il register renaming.
- **WAW (Write After Write)**: due istruzioni scrivono sullo stesso registro. Questa dipendenza "di output" richiede che l'ultima scrittura nel programma originale sia anche l'ultima ad essere effettivamente eseguita.

Le unità di scheduling del processore utilizzano algoritmi sofisticati come il **Tomasulo's algorithm** o il **scoreboarding** per tracciare dinamicamente queste dipendenze. Questi meccanismi mantengono una rappresentazione delle istruzioni in volo e dei loro operandi, permettendo al processore di identificare quando un'istruzione può essere eseguita in sicurezza senza violare le dipendenze del programma.

Register Renaming: Eliminazione delle False Dipendenze

Il **register renaming** è una tecnica fondamentale che permette ai processori di eliminare le dipendenze WAR e WAW, aumentando significativamente il parallelismo disponibile nell'esecuzione fuori ordine. Questa tecnica sfrutta il fatto che i processori dispongono di molti più registri fisici di quanti siano visibili al programmatore attraverso l'architettura ISA (Instruction Set Architecture).

01

Registri Architetturali vs Fisici

L'ISA definisce un insieme limitato di registri architetturali (ad esempio, 32 registri in RISC-V o ARM). Internamente, il processore mantiene un pool molto più ampio di registri fisici (spesso 100-200 o più) che sono invisibili al programmatore.

02

Mapping Dinamico

Durante l'esecuzione, il processore mantiene una tabella di mappatura (Register Alias Table, RAT) che associa dinamicamente ogni registro architetturale a un registro fisico specifico. Quando un'istruzione scrive un registro, viene allocato un nuovo registro fisico.

03

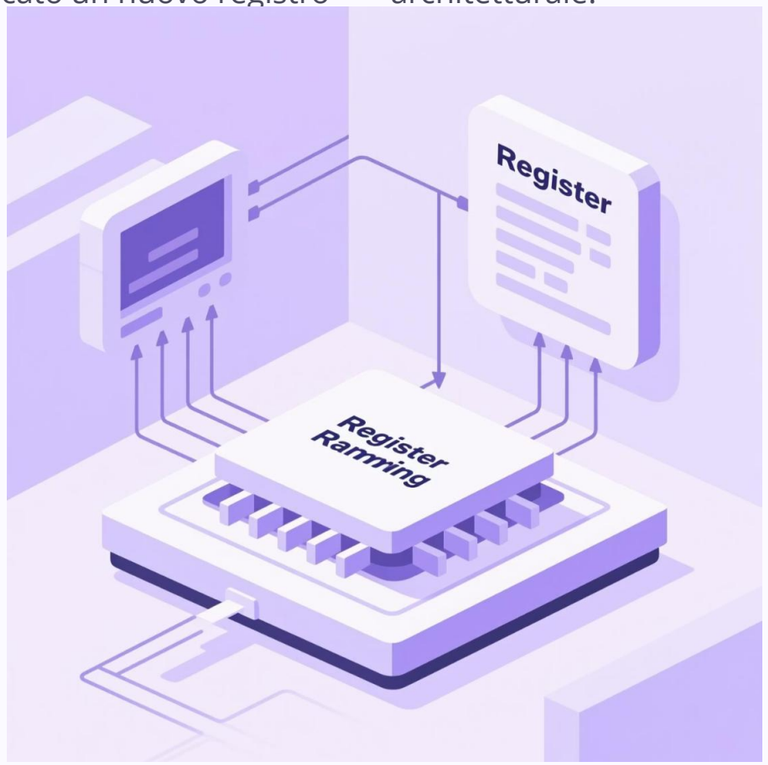
Eliminazione delle False Dipendenze

Allocando nuovi registri fisici per ogni scrittura, le dipendenze WAR e WAW vengono eliminate: istruzioni diverse operano su registri fisici diversi, anche se utilizzano lo stesso nome di registro architetturale.

Esempio Pratico

```
R1 = R2 + R3  # usa P10 per R1
R4 = R1 + R5  # legge P10
R1 = R6 + R7  # usa P11 per R1 (nuovo!)
R8 = R1 + R9  # legge P11
```

In questo esempio, le due scritture a R1 utilizzano registri fisici diversi (P10 e P11), eliminando la dipendenza WAW e permettendo un'esecuzione più parallela.



Ottimizzazione del Commit

Al momento del riordinamento delle istruzioni (commit/retirement), anziché copiare i valori dai registri fisici temporanei ai registri architetturali visibili, il processore **rinomina semplicemente i puntatori** nella tabella di mappatura. Questo aggiornamento della RAT è istantaneo e risparmia il tempo necessario per il trasferimento effettivo dei dati.

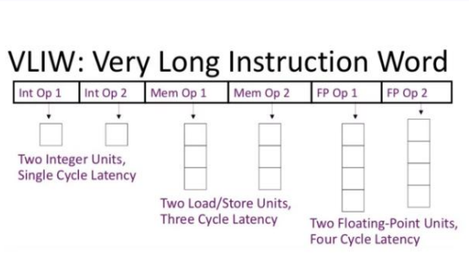
Gestione del Pool di Registri

I registri fisici vengono liberati e restituiti al pool quando non sono più necessari, ovvero quando tutte le istruzioni che potrebbero leggerli hanno completato l'esecuzione. Un meccanismo di **free list** gestisce l'allocazione e il rilascio dei registri fisici.

Supporto al Rollback

Il register renaming facilita anche il ripristino dello stato in caso di branch misprediction o eccezioni: il processore può semplicemente ripristinare una copia precedente della RAT, eliminando istantaneamente tutti i cambiamenti delle istruzioni speculative.

VLIW: Very Long Instruction Word



Tutte le innovazioni architetturali discusse finora—**pipeline, super-scalarità, predicazione ed esecuzione fuori ordine**—convergono verso l'implementazione di un modello computazionale studiato approfonditamente nei centri di ricerca: il **VLIW (Very Long Instruction Word)**. Questo paradigma rappresenta un approccio radicalmente diverso alla parallelizzazione, spostando gran parte della complessità dall'hardware al software.

Parallelismo Hardware

Nelle architetture tradizionali superscalari, l'hardware dinamicamente identifica e sfrutta il parallelismo a runtime attraverso meccanismi complessi come scoreboarding, reservation stations e reorder buffers.

Parallelismo nel Compilatore

Nel modello VLIW, il **compilatore analizza staticamente il codice** e genera istruzioni "pre-cucinate" che specificano esplicitamente quali operazioni possono essere eseguite in parallelo, eliminando la necessità di logica hardware complessa.

Esecuzione Ottimizzata

La CPU VLIW esegue semplicemente le operazioni parallele specificate nell'istruzione, con un hardware molto più semplice e consumando meno energia rispetto alle architetture superscalari out-of-order.

Caratteristiche del Modello VLIW

- **Istruzioni Lunghe**: ogni istruzione VLIW contiene multiple operazioni indipendenti (tipicamente 4-8) che
- possono essere eseguite simultaneamente su unità funzionali diverse.
- **Scheduling Statico**: il compilatore determina a compile-time l'ordine di esecuzione e il binding delle operazioni
- alle unità funzionali, utilizzando tecniche avanzate come software pipelining e trace scheduling.
- **Hardware Semplificato**: l'assenza di logica di scheduling dinamico riduce drasticamente la complessità del processore,
- diminuendo consumo energetico, area del chip e costo di produzione.
- **Predicibilità**: i tempi di esecuzione sono più deterministici, un vantaggio importante per sistemi real-time ed embedded.



Vantaggi del VLIW

Minor consumo energetico, design hardware più semplice, migliore scalabilità con l'aumento delle unità funzionali, e prestazioni prevedibili. Il compilatore ha una visione globale del programma e può applicare ottimizzazioni sofisticate impossibili per l'hardware dinamico.

Sfide e Limitazioni

Il compilatore deve essere estremamente sofisticato per estrarre parallelismo efficacemente. La dimensione del codice aumenta significativamente (molte operazioni NOP quando non c'è parallelismo disponibile). La compatibilità binaria è problematica: il codice compilato per una generazione di processore potrebbe non essere ottimale per la successiva.

Esempi Pratici di VLIW: Le architetture VLIW hanno trovato successo in domini specifici. Intel Itanium (IA-64) ha rappresentato il tentativo più ambizioso di portare VLIW nei server, ma ha avuto successo limitato. Al contrario, i DSP (Digital Signal Processors) di Texas Instruments (TMS320C6x) e i processori Tensilica per applicazioni embedded utilizzano con successo principi VLIW, sfruttando la natura altamente parallela e predicibile dei loro workload target.